

DPRODUCT REQUIREMENTS DOCUMENT

Tugas Akhir Semester — Mata Kuliah Temu Kembali Informasi

Sistem Temu Kembali Dokumen Berbasis Dense Retrieval pada Dokumen Manajemen Energi:

Integrasi Multilingual Sentence-BERT dan Cross-Encoder Reranking

Keterangan	Detail
Program Studi	Informatika
Mata Kuliah	Temu Kembali Informasi (TKI)
Semester	6 (Genap 2024/2025)
Institusi	Universitas Internasional Semen Indonesia (UISI)
Topik Dataset	Manajemen Energi (50 Dokumen Jurnal)
Versi Dokumen	1.0 Final
Status	Siap Implementasi

Daftar Isi

1. Latar Belakang & Pengertian
2. Tujuan & Ruang Lingkup
3. Dataset & Ground Truth
4. Arsitektur Sistem
5. Komponen Teknis: Rumus & Penjelasan
6. Library & Dependensi
7. Pipeline Implementasi Detail
8. Metrik Evaluasi
9. Perbandingan Sistem Lama vs Baru
10. Output & Luaran
11. Risiko & Mitigasi
12. Glosarium

BAB 1 — LATAR BELAKANG & PENGERTIAN

1.1 Latar Belakang

Sistem IR (Information Retrieval) tradisional yang dibangun pada tugas UTS menggunakan pendekatan TF-IDF + Vector Space Model (VSM). Pendekatan ini bekerja berdasarkan kecocokan leksikal (exact term match) artinya dokumen hanya dianggap relevan jika mengandung kata yang sama persis dengan query, setelah proses stemming.

Kelemahan utama pendekatan ini adalah ketidakmampuannya menangkap kemiripan makna (semantic similarity). Misalnya, query "penghematan listrik" tidak akan menemukan dokumen yang membahas "efisiensi konsumsi energi" meskipun keduanya memiliki makna yang sangat mirip, karena tidak ada tumpang tindih kata setelah stemming.

Tugas UAS ini mengembangkan sistem tersebut dengan menggantikan representasi sparse (TF-IDF) menjadi representasi dense (embedding vektor dari model transformer), serta menambahkan tahap reranking menggunakan Cross-Encoder untuk meningkatkan presisi hasil akhir.

1.2 Pengertian Istilah Kunci

1.2.1 Information Retrieval (IR)

Disiplin ilmu yang mempelajari cara menemukan materi (biasanya dokumen teks) yang relevan dengan kebutuhan informasi pengguna dari dalam koleksi dokumen yang besar. Sistem IR menerima query dari pengguna dan mengembalikan daftar dokumen yang diurutkan berdasarkan tingkat relevansinya.

1.2.2 TF-IDF (Term Frequency–Inverse Document Frequency)

Metode statistik untuk mengukur seberapa penting suatu kata terhadap dokumen dalam suatu koleksi. Merupakan pendekatan sparse representation — setiap dokumen direpresentasikan sebagai vektor berisi bobot tiap kata di kamus. Vektor ini sangat berdimensi tinggi namun sebagian besar isinya nol (sparse).

1.2.3 Dense Retrieval

Pendekatan IR modern di mana dokumen dan query masing-masing dikodekan menjadi vektor padat berdimensi tetap (misal 768 dimensi) menggunakan model neural network. Berbeda dengan TF-IDF yang sparse, vektor dense ini merepresentasikan makna semantik teks sehingga dua kalimat bermakna sama akan menghasilkan vektor yang berdekatan di ruang vektor, meskipun kata-katanya berbeda.

1.2.4 Sentence-BERT (SBERT)

Varian dari model BERT yang dimodifikasi menggunakan arsitektur Siamese Network, dilatih khusus untuk menghasilkan representasi kalimat (sentence embedding) yang optimal untuk tugas perbandingan

kemiripan semantik. Kelebihan utama SBERT dibanding BERT biasa adalah kemampuannya menghasilkan embedding kalimat yang bisa langsung dibandingkan menggunakan cosine similarity tanpa perlu forward pass bersama.

1.2.5 Multilingual SBERT (paraphrase-multilingual-mpnet-base-v2)

Model SBERT yang mendukung lebih dari 50 bahasa termasuk Bahasa Indonesia, dilatih pada lebih dari 1 miliar pasangan kalimat paralel lintas bahasa. Menggunakan arsitektur XLM-RoBERTa sebagai backbone dengan pooling layer di atasnya. Menghasilkan embedding berdimensi 768 yang berada dalam ruang semantik yang sama untuk semua bahasa yang didukung.

1.2.6 Bi-Encoder

Arsitektur di mana query dan dokumen masing-masing di-encode secara terpisah dan independen menggunakan model yang sama, menghasilkan dua vektor yang kemudian dibandingkan menggunakan cosine similarity. Sangat cepat karena vektor dokumen bisa di-precompute, tapi sedikit kurang presisi karena tidak ada interaksi langsung antara token query dan dokumen saat encoding.

1.2.7 Cross-Encoder

Arsitektur di mana query dan dokumen digabung menjadi satu input dan diproses bersama oleh model transformer. Output berupa skor relevansi tunggal per pasangan (query, dokumen). Jauh lebih presisi dari bi-encoder karena attention mechanism dapat menangkap interaksi antar token query dan dokumen secara langsung, tapi tidak skalabel untuk retrieval awal karena harus dijalankan per pasangan.

1.2.8 Retrieve-Then-Rerank Pipeline

Strategi dua tahap yang menggabungkan kelebihan bi-encoder (cepat) dan cross-encoder (presisi). Tahap 1: bi-encoder menyaring kandidat top-k dari seluruh koleksi secara efisien. Tahap 2: cross-encoder menilai ulang (rerank) hanya k kandidat tersebut untuk menghasilkan ranking final yang lebih akurat. Pipeline ini adalah standar industri yang dipakai oleh sistem pencarian modern seperti Google dan Bing.

1.2.9 FAISS (Facebook AI Similarity Search)

Library open-source dari Meta AI untuk indexing dan pencarian vektor berdimensi tinggi secara efisien. FAISS membangun struktur data index dari kumpulan vektor dokumen, kemudian saat ada query baru dapat dengan cepat menemukan vektor paling mirip. Untuk dataset kecil (50 dokumen) menggunakan IndexFlatIP (exact search dengan inner product), yang ekuivalen dengan cosine similarity setelah normalisasi L2.

1.2.10 Cosine Similarity

Ukuran kemiripan antara dua vektor berdasarkan sudut di antara keduanya, bukan jarak euclidean. Nilai berkisar dari -1 (berlawanan arah) hingga 1 (arah identik). Dalam IR, skor 1 berarti dokumen sangat relevan dengan query, skor 0 berarti tidak ada kemiripan. Dipilih sebagai metrik kemiripan karena tidak terpengaruh panjang dokumen (magnitude vektor tidak berpengaruh).

BAB 2 — TUJUAN & RUANG LINGKUP

2.1 Tujuan Utama

1. Mengupgrade sistem IR tradisional (TF-IDF + VSM) menjadi sistem IR modern berbasis dense retrieval yang mampu memahami makna semantik query dan dokumen.
2. Mengintegrasikan Multilingual Sentence-BERT sebagai bi-encoder untuk menghasilkan representasi vektor dense dari seluruh 50 dokumen jurnal manajemen energi.
3. Mengintegrasikan Cross-Encoder (mMiniLM Multilingual) sebagai reranker untuk meningkatkan presisi ranking pada top-k hasil retrieval.
4. Membandingkan performa sistem lama (TF-IDF baseline) dengan sistem baru (SBERT + Cross-Encoder) menggunakan metrik evaluasi IR standar: MAP, MRR, dan NDCG.
5. Menghasilkan laporan penelitian mini dalam format paper ilmiah beserta source code yang dapat direproduksi di Google Colab.

2.2 Ruang Lingkup (Scope)

2.2.1 Yang Termasuk dalam Scope

- Dataset: 50 dokumen jurnal yang sama persis dengan tugas UTS (tidak boleh berubah)
- Bahasa dokumen: Bahasa Indonesia (dan sebagian abstrak bahasa Inggris)
- Ground truth: Minimal 5 query dengan relevance judgment manual
- Sistem baseline: TF-IDF + VSM dari kode `tfidf_vsm.py` yang sudah ada
- Sistem modern: SBERT Multilingual + FAISS + Cross-Encoder mMiniLM
- Evaluasi: MAP@10, MRR@10, NDCG@10
- Platform implementasi: Google Colab (Python, GPU T4 gratis)
- Luaran: Notebook Colab + Laporan PDF format paper ilmiah

2.2.2 Yang Tidak Termasuk dalam Scope

- Fine-tuning model SBERT atau Cross-Encoder pada dataset manajemen energi
- Penambahan dokumen baru di luar 50 dokumen yang sudah ada
- Deployment ke production server atau web application
- Integrasi dengan database eksternal atau API pihak ketiga
- Pembangunan antarmuka Streamlit baru (fokus pada notebook evaluasi)

BAB 3 — DATASET & GROUND TRUTH

3.1 Dataset

Dataset yang digunakan adalah file CSV yang sama dari tugas UTS:

- **File:** TKI_Keyword_Manajemen Energi(Jurnal).csv
- **Jumlah dokumen:** 50 dokumen jurnal ilmiah
- **Domain:** Manajemen Energi (energy management, renewable energy, energy efficiency, dsb)
- **Bahasa:** Bahasa Indonesia dengan beberapa abstrak dalam Bahasa Inggris
- **Format kolom:** DocID, Judul, Abstrak/Konten (sesuai struktur file asli)

⚠ Penting: Gunakan teks dokumen versi MINIMAL cleaning untuk embedding SBERT — bukan hasil stemming Sastrawi.

Alasan: Model transformer dilatih pada teks natural, bukan kata dasar. Stemming merusak konteks semantik yang dibutuhkan model.

3.2 Versi Teks yang Digunakan per Sistem

Sistem	Versi Teks yang Dipakai	Alasan
TF-IDF Baseline	Hasil stemming Sastrawi (dari pipeline UTS)	Sesuai implementasi lama, tidak diubah
SBERT Bi-Encoder	Teks asli / lowercase saja tanpa stemming	Model transformer butuh teks natural
Cross-Encoder Reranker	Teks asli / lowercase saja tanpa stemming	Sama dengan input SBERT

3.3 Ground Truth Queries

Minimal 5 query harus didefinisikan secara manual beserta relevance judgment-nya. Query harus bervariasi dalam tingkat kesulitan leksikal untuk menunjukkan perbedaan kemampuan TF-IDF vs semantic search:

ID Query	Teks Query	Jenis Kesulitan	Alasan Pemilihan
Q1	efisiensi energi industri manufaktur	Mudah (lexical match)	Term kemungkinan muncul langsung di dokumen
Q2	penghematan listrik gedung perkantoran	Sedang	Sinonim parsial — 'penghematan' vs 'efisiensi'
Q3	kebijakan transisi menuju energi bersih	Sulit (semantic)	Parafrase — 'energi bersih' vs 'energi terbarukan'
Q4	audit penggunaan daya pada sistem industri	Sulit (semantic)	Berbeda kata tapi semakna dengan 'manajemen konsumsi'
Q5	optimalisasi biaya operasional pembangkit listrik	Sangat sulit	Framing ekonomi dari topik manajemen energi

3.4 Cara Membuat Relevance Judgment

Untuk setiap query di atas, lakukan penilaian relevansi terhadap seluruh 50 dokumen secara manual:

1. Baca judul dan abstrak tiap dokumen.
2. Beri skor relevansi menggunakan skala berikut:
 - Skor 0 = Tidak relevan sama sekali
 - Skor 1 = Agak relevan (menyinggung topik tapi bukan fokus utama)
 - Skor 2 = Relevan (membahas topik yang dimaksud query)
 - Skor 3 = Sangat relevan (dokumen inti yang menjawab query secara langsung)
3. Simpan dalam format CSV: query_id, doc_id, relevance_score
4. Untuk MAP dan MRR, binarisasi: skor ≥ 1 dianggap relevan (1), skor 0 tetap 0.
5. Untuk NDCG, gunakan skor graded (0–3) langsung.

BAB 4 — ARSITEKTUR SISTEM

4.1 Gambaran Umum Pipeline

Sistem baru menggunakan pipeline dua tahap (retrieve-then-rerank) yang bekerja sebagai berikut:

Tahap	Komponen	Input	Output
Offline (satu kali)	SBERT Encoding	50 dokumen teks mentah	50 vektor embedding (50×768)
Offline (satu kali)	FAISS Indexing	50 vektor embedding	FAISS index tersimpan di disk
Online (per query)	Query Encoding	Teks query pengguna	1 vektor query (1×768)
Online (per query)	FAISS Retrieval	Vektor query + FAISS index	Top-k kandidat (misal $k=10$)
Online (per query)	Cross-Encoder Reranking	Pasangan (query, dok) $\times k$	Skor relevansi per pasangan
Online (per query)	Final Ranking	Skor Cross-Encoder	Daftar dokumen terurut final

4.2 Komponen per Modul

4.2.1 Modul Preprocessing (untuk SBERT)

- Lowercase seluruh teks
- Hapus karakter khusus yang tidak bermakna (simbol, angka opsional)
- Tidak dilakukan stemming — teks dibiarkan natural
- Gabungkan kolom Judul + Abstrak menjadi satu string per dokumen

4.2.2 Modul Embedding (Bi-Encoder)

- **Model:** sentence-transformers/paraphrase-multilingual-mpnet-base-v2
- **Dimensi output:** 768 float per dokumen/query
- **Normalisasi:** L2 normalization sebelum masuk ke FAISS IndexFlatIP
- **Batch size:** 16–32 (sesuaikan dengan RAM Colab)
- **Device:** GPU (cuda) jika tersedia, fallback ke CPU

4.2.3 Modul Vector Index (FAISS)

- **Index type:** IndexFlatIP (Inner Product — ekuivalen cosine setelah L2 norm)
- **Dimensi:** 768 (harus sama dengan dimensi embedding SBERT)
- **Operasi:** `faiss.normalize_L2()` → `index.add()` → `index.search(query_vec, k)`
- **Storage:** Simpan ke disk dengan `faiss.write_index()` untuk efisiensi

4.2.4 Modul Reranker (Cross-Encoder)

- **Model:** cross-encoder/mmarco-mMiniLMv2-L12-H384-v1
- **Input:** List of tuples [(query, doc_text), ...]
- **Output:** List of float scores (satu skor per pasangan)
- **k kandidat:** 10–20 dokumen top dari FAISS (disesuaikan)

4.2.5 Modul Evaluasi

- Hitung MAP@10, MRR@10, NDCG@10 untuk tiga sistem: TF-IDF, SBERT-only, SBERT+Reranking
- Input: ranking per query + ground truth relevance judgment
- Output: tabel perbandingan skor + analisis kualitatif per query

BAB 5 — RUMUS & PENJELASAN TEKNIS

5.1 Rumus Sistem Lama (Baseline)

5.1.1 Term Frequency — Log Weighting

Rumus TF yang dipakai di sistem lama menggunakan log frequency weighting untuk mengurangi efek dominasi term yang sangat sering muncul:

$$\text{TF}(t, d) = 1 + \log_{10}(\text{raw_tf}(t, d)) \quad \text{jika raw_tf} > 0, \text{ else } 0$$

di mana $\text{raw_tf}(t, d)$ = jumlah kemunculan mentah term t dalam dokumen d .

5.1.2 Inverse Document Frequency

$$\text{IDF}(t) = \log_{10}(N / \text{df}_t)$$

di mana N = total dokumen (50), df_t = jumlah dokumen yang mengandung term t . Term yang muncul di banyak dokumen mendapat IDF rendah (kurang diskriminatif).

5.1.3 Bobot TF-IDF

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \text{IDF}(t)$$

5.1.4 Cosine Similarity (VSM)

$$\text{sim}(q, d) = (q \cdot d) / (|q| \times |d|)$$

di mana q = vektor TF-IDF query, d = vektor TF-IDF dokumen. Semakin mendekati 1, semakin mirip.

5.2 Rumus Sistem Baru (Dense Retrieval)

5.2.1 Mean Pooling — Menghasilkan Sentence Embedding dari Token Embeddings

SBERT menghasilkan embedding per-token. Untuk mendapatkan satu vektor kalimat, dilakukan mean pooling dengan memperhitungkan attention mask:

$$\text{sentence_emb} = \sum(\text{token_emb}_i \times \text{mask}_i) / \sum(\text{mask}_i)$$

di mana $\text{mask}_i = 1$ untuk token nyata, 0 untuk padding token. Ini mencegah token padding mempengaruhi representasi kalimat.

5.2.2 L2 Normalization (sebelum FAISS IndexFlatIP)

$$\text{v_norm} = v / \|v\|_2 \quad \text{di mana } \|v\|_2 = \sqrt{\sum v_i^2}$$

Setelah normalisasi, inner product (dot product) antara dua vektor menjadi ekuivalen dengan cosine similarity, sehingga IndexFlatIP (inner product) di FAISS menghasilkan skor yang sama dengan cosine similarity.

5.2.3 Cosine Similarity (Dense Space)

$$\text{score_bi}(q, d) = \text{emb}(q) \cdot \text{emb}(d) \quad [\text{setelah normalisasi L2}]$$

di mana $\text{emb}(q)$ dan $\text{emb}(d)$ adalah vektor embedding hasil SBERT yang sudah dinormalisasi.

5.2.4 Cross-Encoder Scoring

Cross-encoder menerima konkatenasi [CLS] + query + [SEP] + dokumen + [SEP] sebagai satu input, dan menghasilkan skor relevansi melalui linear layer di atas representasi [CLS]:

$$\text{score_ce}(q, d) = W \cdot h_{[\text{CLS}]}(\text{concat}[q, d]) + b$$

di mana $h_{[\text{CLS}]}$ adalah representasi token [CLS] dari output transformer, W adalah weight matrix linear layer, b adalah bias. Skor ini yang dipakai sebagai dasar reranking final.

5.3 Rumus Metrik Evaluasi

5.3.1 Precision@k

$$P@k = (\text{jumlah dokumen relevan dalam top-k}) / k$$

5.3.2 Average Precision (AP) per Query

$$AP(q) = \sum_k [P@k \times \text{rel}(k)] / |\text{Rel}_q|$$

di mana $\text{rel}(k) = 1$ jika dokumen di posisi k relevan, 0 jika tidak. $|\text{Rel}_q|$ = total dokumen relevan untuk query q .

5.3.3 Mean Average Precision (MAP)

$$MAP = (1/|Q|) \times \sum_q AP(q)$$

di mana $|Q|$ = jumlah query (minimal 5). MAP mengukur rata-rata presisi di semua query, memperhitungkan urutan dokumen relevan dalam ranking.

5.3.4 Mean Reciprocal Rank (MRR)

$$MRR = (1/|Q|) \times \sum_q (1 / \text{rank}_q)$$

di mana rank_q = posisi dokumen relevan pertama untuk query q . Mengukur seberapa cepat sistem menemukan dokumen relevan pertama.

5.3.5 Normalized Discounted Cumulative Gain (NDCG@k)

$$\text{DCG}@k = \sum_{i=1}^k (2^{\text{rel}_i} - 1) / \log_2(i+1)$$

$$\text{NDCG}@k = \text{DCG}@k / \text{IDCG}@k$$

di mana rel_i = skor relevansi graded (0–3) dokumen di posisi i , $\text{IDCG}@k$ = DCG ideal (jika dokumen diurutkan sempurna dari paling relevan ke paling tidak relevan). NDCG mendukung penilaian graded relevance, sehingga membedakan 'sangat relevan' dan 'agak relevan'.

BAB 6 — LIBRARY & DEPENDENSI

6.1 Tabel Dependensi Lengkap

Library	Versi Min.	Fungsi dalam Sistem	Cara Install
sentence-transformers	>=2.2.0	Memuat & menjalankan SBERT bi-encoder dan cross-encoder reranker	pip install sentence-transformers
faiss-cpu	>=1.7.0	Membangun index vektor dan melakukan pencarian nearest-neighbor	pip install faiss-cpu
transformers	>=4.30.0	Backend HuggingFace untuk model BERT/XLM-R (diperlukan oleh sentence-transformers)	pip install transformers
torch	>=2.0.0	Framework deep learning untuk inferensi model transformer	pip install torch
numpy	>=1.21.0	Operasi matriks dan vektor (normalisasi L2, dll)	pip install numpy
pandas	>=1.3.0	Membaca CSV dataset, menyimpan hasil evaluasi ke tabel	pip install pandas
scikit-learn	>=1.0.0	Metrik evaluasi IR (ndcg_score), TF-IDF baseline comparison	pip install scikit-learn
Sastrawi	>=1.0.1	Stemming Bahasa Indonesia untuk sistem baseline TF-IDF	pip install PySastrawi
tqdm	>=4.0.0	Progress bar saat encoding dokumen dalam batch	pip install tqdm
matplotlib	>=3.5.0	Visualisasi grafik perbandingan MAP/MRR/NDCG antar sistem	pip install matplotlib
seaborn	>=0.11.0	Heatmap relevance judgment dan visualisasi tambahan	pip install seaborn

6.2 Model yang Diunduh Otomatis dari HuggingFace

Model	Ukuran	Dipakai untuk	Model ID HuggingFace
Multilingual SBERT	~278 MB	Bi-encoder embedding dokumen & query	sentence-transformers/paraphrase-multilingual-mpnet-base-v2
mMiniLM Cross-Encoder	~117 MB	Reranking top-k kandidat	cross-encoder/mmarco-mMiniLMv2-L12-H384-v1

i Di Google Colab: model diunduh sekali lalu di-cache. Saat runtime restart, download ulang (~5–10 menit).

Gunakan GPU runtime (Runtime → Change runtime type → T4 GPU) untuk encoding lebih cepat.

BAB 7 — PIPELINE IMPLEMENTASI DETAIL

7.1 Struktur Notebook

Seluruh implementasi ditulis dalam satu Google Colab Notebook dengan struktur sel sebagai berikut:

Sel / Bagian	Konten	Output
Sel 1	Instalasi library (pip install)	Semua dependensi terpasang
Sel 2	Import library dan konfigurasi global	Variabel PATH, K, device
Sel 3	Load dataset CSV + preprocessing minimal	DataFrame 50 dokumen, kolom teks_clean
Sel 4	Reproduksi baseline TF-IDF pada 5 query	Dict {query_id: ranked_doc_list}
Sel 5	SBERT encoding 50 dokumen (offline)	Matrix embeddings (50, 768)
Sel 6	Build FAISS index + simpan ke disk	File faiss_index.bin
Sel 7	SBERT retrieval pada 5 query (bi-encoder)	Dict {query_id: top_k_candidates}
Sel 8	Cross-Encoder reranking per query	Dict {query_id: reranked_list}
Sel 9	Load ground truth relevance judgment	DataFrame qrels (query_id, doc_id, relevance)
Sel 10	Hitung MAP, MRR, NDCG untuk 3 sistem	Tabel skor komparatif
Sel 11	Visualisasi (grafik bar, analisis kualitatif)	Plots & tabel markdown
Sel 12	Simpan semua hasil ke CSV	File hasil_evaluasi.csv

7.2 Detail Implementasi per Komponen

7.2.1 Preprocessing Teks untuk SBERT

Langkah preprocessing minimal yang diperlukan sebelum SBERT encoding:

1. Gabungkan kolom judul dan abstrak/konten: `text = judul + ' ' + konten`
2. Lowercase: `text = text.lower()`
3. Hapus karakter non-alfabet dan non-spasi yang tidak bermakna (opsional)

4. Truncate jika lebih dari 512 token — SBERT memiliki batas panjang input
5. JANGAN lakukan stemming — ini langkah kritis yang membedakan preprocessing SBERT vs TF-IDF

7.2.2 SBERT Encoding (Offline Phase)

Fase ini hanya dijalankan sekali dan hasilnya disimpan:

1. Load model: `SentenceTransformer('paraphrase-multilingual-mpnet-base-v2')`
2. Encode semua dokumen: `model.encode(docs, batch_size=16, show_progress_bar=True, normalize_embeddings=True)`
3. Parameter `normalize_embeddings=True` otomatis melakukan L2 normalization
4. Simpan embeddings: `np.save('doc_embeddings.npy', embeddings)`

7.2.3 FAISS Index Building

1. Buat index: `index = faiss.IndexFlatIP(768)` ← 768 = dimensi embedding
2. Tambahkan vektor: `index.add(embeddings_float32)` ← pastikan dtype float32
3. Simpan: `faiss.write_index(index, 'faiss_index.bin')`
4. Load saat dibutuhkan: `faiss.read_index('faiss_index.bin')`

7.2.4 Query Retrieval (Bi-Encoder Phase)

1. Encode query: `q_emb = model.encode([query_text], normalize_embeddings=True)`
2. Search FAISS: `scores, indices = index.search(q_emb, k=20)` ← ambil top-20
3. scores berisi skor cosine similarity, indices berisi posisi dokumen di array
4. Petakan indices ke `doc_id` menggunakan dataframe dokumen

7.2.5 Cross-Encoder Reranking

1. Load reranker: `from sentence_transformers import CrossEncoder; reranker = CrossEncoder('cross-encoder/mmarco-mMiniLMv2-L12-H384-v1')`
2. Buat pasangan input: `pairs = [(query, doc_texts[i]) for i in top_20_indices]`
3. Score: `ce_scores = reranker.predict(pairs)`
4. Urutkan ulang: `reranked = sorted(zip(ce_scores, top_20_indices), reverse=True)`
5. Ambil top-10 sebagai hasil akhir sistem modern

7.2.6 Evaluasi MAP, MRR, NDCG

Implementasi menggunakan fungsi manual atau `sklearn.metrics.ndcg_score`:

1. Load qrels (ground truth) dari file CSV relevance judgment
2. Untuk setiap query, konversi ranked list ke binary relevance list (urutan sesuai ranking)
3. Hitung AP per query menggunakan loop atas posisi ranking
4. Hitung MRR: cari rank_first_relevant per query
5. Hitung NDCG: gunakan `sklearn.metrics.ndcg_score(true_scores, pred_scores, k=10)`
6. Rata-ratakan semua query untuk mendapat MAP, MRR, NDCG final
7. Ulangi untuk ketiga sistem: TF-IDF, SBERT-only, SBERT+Reranking

BAB 8 — METRIK EVALUASI

8.1 Metrik yang Digunakan

Metrik	Definisi Singkat	Untuk Apa	Skala Relevansi
MAP@10	Rata-rata Average Precision di semua query pada top-10	Mengukur kualitas ranking secara keseluruhan	Biner (0/1)
MRR@10	Rata-rata Reciprocal Rank dokumen relevan pertama	Mengukur seberapa cepat dok relevan pertama ditemukan	Biner (0/1)
NDCG@10	Normalized Discounted Cumulative Gain pada top-10	Mengukur kualitas ranking dengan graded relevance	Graded (0–3)

8.2 Interpretasi Skor

Rentang Skor	Interpretasi Kualitas Sistem
0.00 – 0.30	Sistem buruk — hasil ranking tidak lebih baik dari random
0.31 – 0.50	Sistem cukup — ada keteraturan tapi relevansi sering tidak di posisi atas
0.51 – 0.70	Sistem baik — mayoritas dokumen relevan di posisi atas ranking
0.71 – 1.00	Sistem sangat baik — hampir semua dokumen relevan berada di posisi teratas

i Pada dataset 50 dokumen dengan 5 query, skor absolut tidak terlalu penting yang penting adalah peningkatan relatif sistem baru terhadap baseline TF-IDF dan konsistensi dengan analisis kualitatif.

BAB 9 — PERBANDINGAN SISTEM LAMA VS BARU

9.1 Tabel Perbandingan Teknis

Aspek	Sistem Lama (TF-IDF)	Sistem Baru (SBERT + Reranking)
Representasi dokumen	Vektor sparse (TF-IDF weights per term)	Vektor dense 768 dimensi
Pemahaman semantik	Tidak ada — hanya exact term match	Ada — sinonim & parafrase tertangkap
Model ML	Tidak ada (pure statistics)	Bi-encoder + Cross-Encoder transformer
Basis data vektor	Tidak ada — cosine similarity on-the-fly	FAISS IndexFlatIP
Waktu retrieval	Sangat cepat (~ms, tanpa preprocessing)	Sedang (~1-3 detik per query dengan GPU)
Waktu index build	Sangat cepat	Lambat sekali (~30-60 detik untuk 50 dok)
Resource yang dibutuhkan	CPU, RAM minimal	GPU direkomendasikan, RAM ~2GB
Interpretability	Tinggi — skor TF-IDF bisa dijelaskan per term	Rendah — black box neural network
Handling sinonim	Tidak bisa	Bisa (fitur utama SBERT)
Handling parafrase	Tidak bisa	Bisa (fitur utama SBERT)
Bahasa Indonesia	Baik dengan Sastrawi stemmer	Baik tanpa stemming (XLM-R multilingual)

9.2 Kapan Sistem Lama Masih Unggul

- Query menggunakan istilah teknis sangat spesifik yang persis sama dengan teks dokumen
- Resource komputasi sangat terbatas (tidak ada GPU, RAM kecil)
- Dataset sangat kecil dan query sederhana tanpa kebutuhan pemahaman semantik
- Dibutuhkan transparansi penuh kenapa dokumen tertentu muncul di hasil

9.3 Kapan Sistem Baru Unggul

- Query menggunakan kata berbeda tapi bermakna sama dengan konten dokumen
- Pengguna memasukkan query dalam gaya bahasa berbeda dari teks jurnal
- Dokumen relevan tidak mengandung kata-kata eksak dari query
- Dibutuhkan pemahaman konteks dan makna, bukan sekadar pencocokan kata

BAB 10 — OUTPUT & LUARAN

10.1 Luaran Teknis (Artefak Sistem)

File	Deskripsi	Format
doc_embeddings.npy	Matrix embedding 50 dokumen hasil SBERT (50×768)	NumPy array
faiss_index.bin	FAISS index yang sudah dibangun dari embedding dokumen	Binary FAISS
qrels.csv	Ground truth relevance judgment (query_id, doc_id, score)	CSV
hasil_evaluasi.csv	Tabel skor MAP, MRR, NDCG tiga sistem per query	CSV
notebook_final.ipynb	Google Colab notebook lengkap dengan semua sel tereksekusi	Jupyter Notebook

10.2 Luaran Akademis

Luaran	Format	Isi Minimum
Laporan Paper Ilmiah	PDF (IEEE atau LNCS format)	Pendahuluan, Metode, Eksperimen/Analisis, Kesimpulan, Referensi
Source Code / Notebook	Google Colab / GitHub	Notebook tereksekusi penuh + README
Presentasi	PowerPoint / Google Slides	Latar belakang, metode, hasil evaluasi, demo, kesimpulan

10.3 Struktur Laporan Paper

1. Pendahuluan — Latar belakang, masalah TF-IDF, tujuan upgrade, kontribusi
2. Tinjauan Pustaka — Dense retrieval, SBERT, cross-encoder, metrik IR
3. Metode — Dataset, pipeline sistem, model yang dipakai, cara evaluasi
4. Hasil Eksperimen — Tabel MAP/MRR/NDCG tiga sistem, analisis kualitatif per query
5. Analisis & Diskusi — Interpretasi hasil, kapan sistem baru unggul, keterbatasan
6. Kesimpulan — Ringkasan temuan, rekomendasi penggunaan, saran penelitian lanjutan
7. Referensi — Paper SBERT (Reimers 2019), LaBSE, FAISS, MS MARCO, paper IR relevan

BAB 11 — RISIKO & MITIGASI

11.1 Tabel Risiko

Risiko	Dampak	Probabilitas	Mitigasi
Colab session timeout saat encoding	Tinggi — harus mulai dari awal	Sedang	Simpan embeddings ke file .npy segera setelah encoding selesai; mount Google Drive
Model tidak terdownload (koneksi lambat)	Sedang — waktu terhambat	Rendah	Pastikan koneksi stabil; gunakan !wget dengan retry
Skor sistem baru lebih rendah dari TF-IDF	Sedang — hasil tidak sesuai harapan	Rendah	Ini tetap valid sebagai temuan ilmiah; analisis kenapa (mungkin query terlalu leksikal)
Ground truth bias (relevance judgment tidak konsisten)	Tinggi — metrik tidak valid	Sedang	Lakukan penilaian dua putaran; dokumentasikan kriteria relevance judgment secara eksplisit
Cross-encoder terlalu lambat di CPU	Rendah — hanya untuk 20 dok per query	Rendah	Gunakan GPU runtime Colab T4; atau kurangi k dari 20 ke 10
Dataset tidak bisa diakses dari repo GitHub	Tinggi — tidak bisa mulai	Rendah	Upload manual ke Colab atau Google Drive; simpan backup

BAB 12 — GLOSARIUM

Istilah	Definisi
Attention Mechanism	Mekanisme dalam transformer yang memungkinkan setiap token memperhatikan token lain dalam konteks yang sama
Baseline	Sistem lama (TF-IDF) yang dijadikan pembanding performa
BERT	Bidirectional Encoder Representations from Transformers — model bahasa dari Google yang menjadi dasar banyak model NLP modern
Bi-Encoder	Arsitektur di mana query dan dokumen di-encode terpisah menjadi dua vektor independen
CLS Token	Token khusus di awal input BERT yang representasinya dipakai sebagai representasi keseluruhan kalimat
Corpus	Kumpulan seluruh dokumen dalam koleksi yang ditelusuri (dalam konteks ini: 50 jurnal)
Cross-Encoder	Arsitektur di mana query dan dokumen diproses bersama dalam satu forward pass transformer
Dense Vector	Vektor berdimensi tetap di mana hampir semua elemen bernilai non-zero (lawan dari sparse)
Embedding	Representasi numerik berupa vektor yang menangkap makna semantik teks
FAISS	Facebook AI Similarity Search — library untuk indexing dan pencarian vektor efisien
Ground Truth	Label relevansi yang dibuat manual sebagai standar kebenaran untuk evaluasi
Inverted Index	Struktur data yang memetakan setiap term ke daftar dokumen yang mengandungnya
L2 Normalization	Proses membagi vektor dengan panjang/magnitudonya sehingga panjang vektor menjadi 1
MAP	Mean Average Precision — metrik evaluasi IR rata-rata dari Average Precision semua query
Mean Pooling	Mengambil rata-rata semua token embedding untuk menghasilkan satu sentence embedding

Istilah	Definisi
MRR	Mean Reciprocal Rank — metrik evaluasi yang mengukur posisi dokumen relevan pertama
NDCG	Normalized Discounted Cumulative Gain — metrik evaluasi IR yang mendukung relevansi bertingkat
Reranking	Proses mengurutkan ulang daftar kandidat yang sudah ada menggunakan model yang lebih presisi
Retrieve-Then-Rerank	Pipeline dua tahap: tahap 1 retrieval cepat, tahap 2 reranking presisi
SBERT	Sentence-BERT — varian BERT yang dioptimalkan untuk menghasilkan sentence embedding
Semantic Search	Pencarian berdasarkan makna/semantik, bukan kecocokan kata eksak
Sentence Embedding	Representasi vektor tunggal yang merangkum makna keseluruhan kalimat/paragraf
Sparse Vector	Vektor di mana sebagian besar elemennya bernilai nol (seperti vektor TF-IDF)
TF-IDF	Term Frequency-Inverse Document Frequency — metode pembobotan term klasik dalam IR
Transformer	Arsitektur neural network berbasis attention mechanism yang menjadi fondasi model NLP modern
Vector Space Model (VSM)	Model representasi dokumen sebagai titik dalam ruang vektor multidimensi
XLM-RoBERTa	Model BERT multibahasa dari Facebook yang menjadi backbone paraphrase-multilingual-mpnet

Dokumen ini dibuat sebagai panduan implementasi untuk tugas UAS Temu Kembali Informasi.

Versi 1.0 — Sistem Temu Kembali Dense Retrieval, Manajemen Energi, UIIS 2025